



Tema 17: Deep Learning II

Contenidos

- *Transformers*.
- Arquitecturas basadas en *Transformers*.
- Grandes modelos del lenguaje.
- Casos de uso y ejemplos.

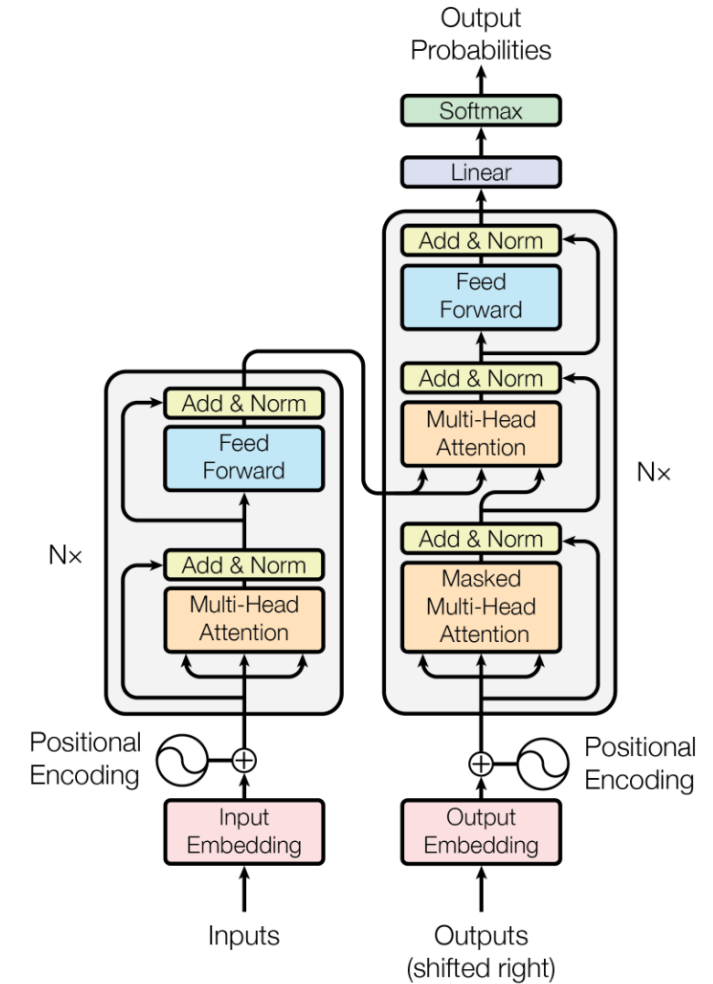


Transformers



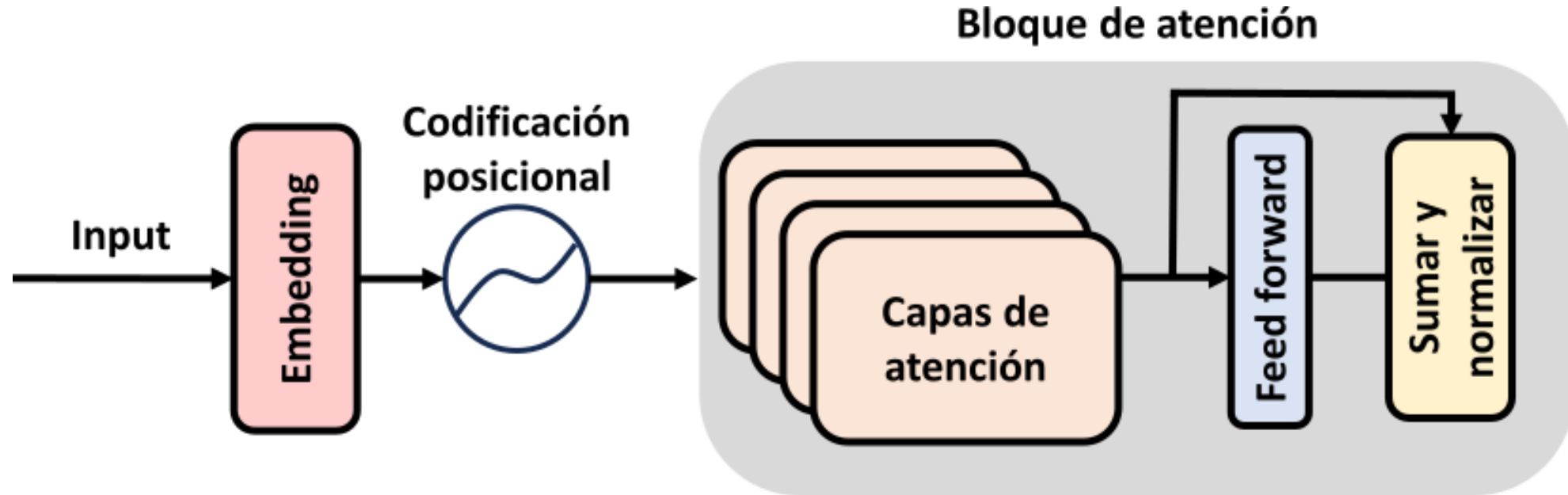
Attention is all you need

- Los mecanismos de atención se plantearon inicialmente como adición a los modelos tipo LSTM.
- En 2018, investigadores de *Google* sacaron un *paper* con una arquitectura en la que se eliminaba la recurrencia.
- Modelos con gran capacidad de captar relaciones entre datos complejos. Habitualmente con muchos millones de parámetros.
- Abanico enorme de aplicaciones con datos de diversa naturaleza (texto, vídeo, audio...).



Elementos principales de un *transformer*

- Espacio de *embeddings*.
- Codificación posicional.
- Autoatención con múltiples capas.

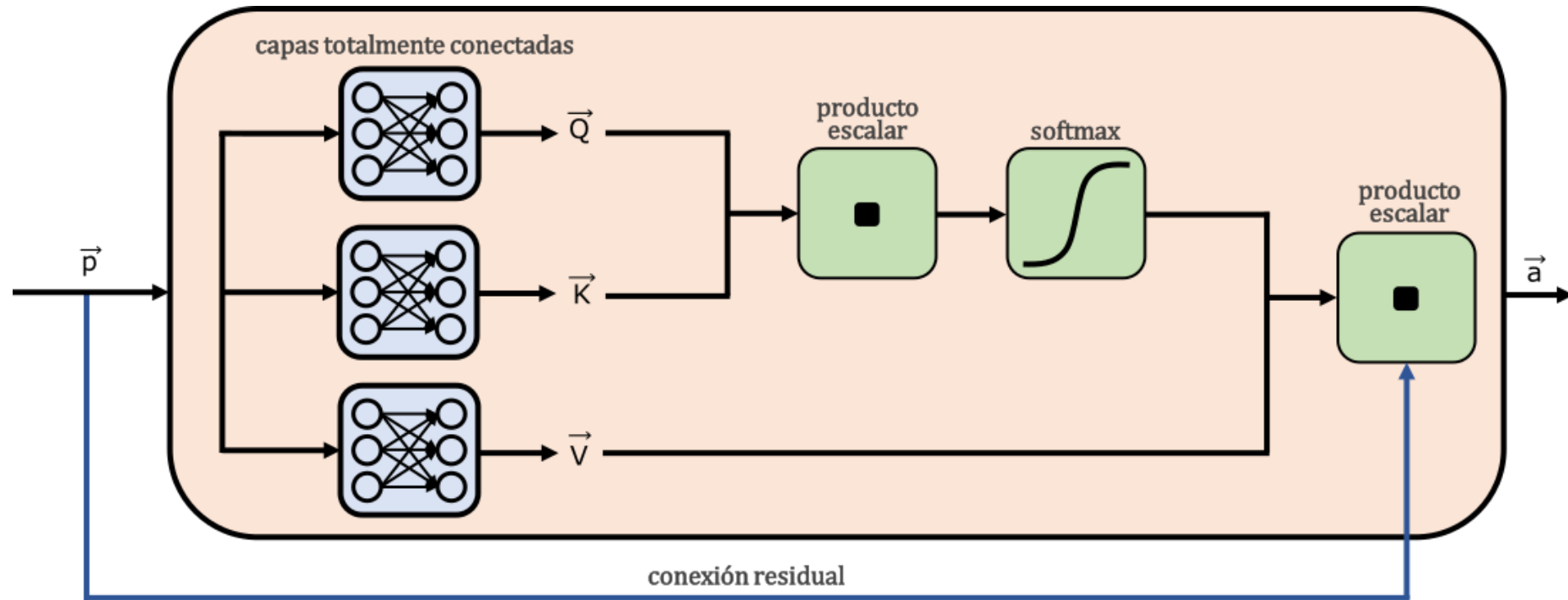


“El gato pardo que estaba en el tejado de mi casa maulló.”

¿Quién maulló?



La clave: vectores *query* – *key* – *value*. Extraen relaciones entre los *tokens* de entrada.



- *Query* → lo que busco.
- *Key* → lo que tengo.
- *Value* → la información

Busco algo (*query*), comparo con claves (*keys*)
y recupero información (*values*)



- Podemos visualizar cómo actuarían los vectores de atención en una frase.
- Cada palabra mira al resto.

Me **encantan** los perros. Los prefiero a los gatos.

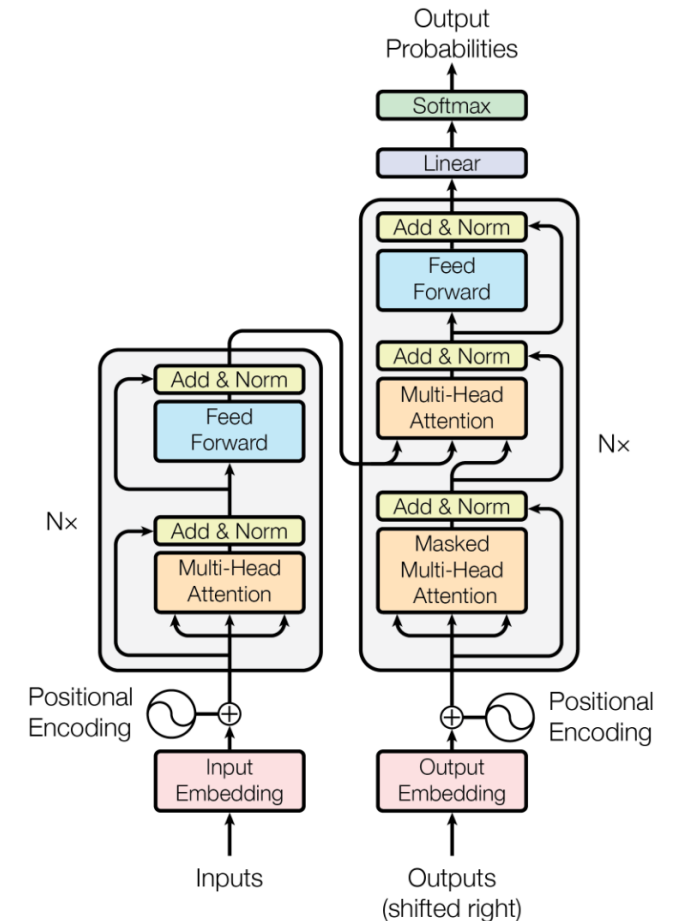
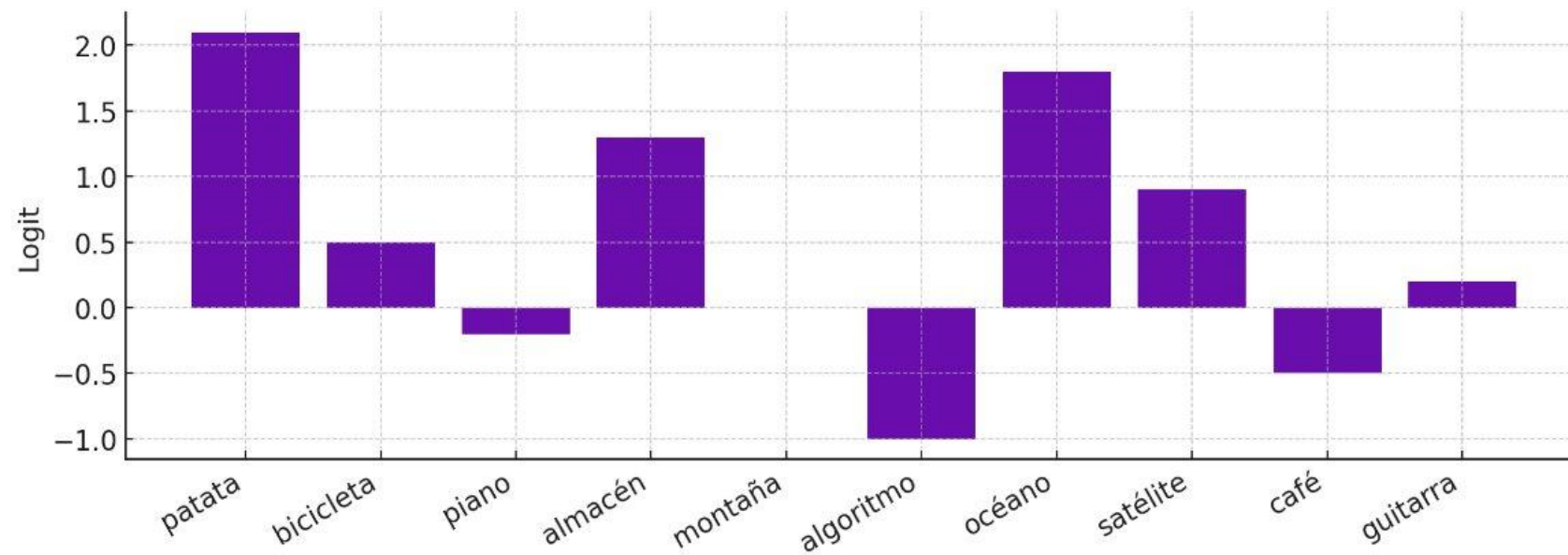
A diagram illustrating attention weights for the word "encantan". Green curved arrows originate from "encantan" and point to each word in the sentence: "Me", "encantan", "los", "perros.", "Los", "prefiero", "a", "los", and "gatos.". The arrows vary in thickness, representing the strength of the attention. The thickest arrow points to "encantan" itself, and other thick arrows point to "perros." and "gatos.", indicating high attention to these words.

Me encantan los perros. **Los** prefiero a los gatos

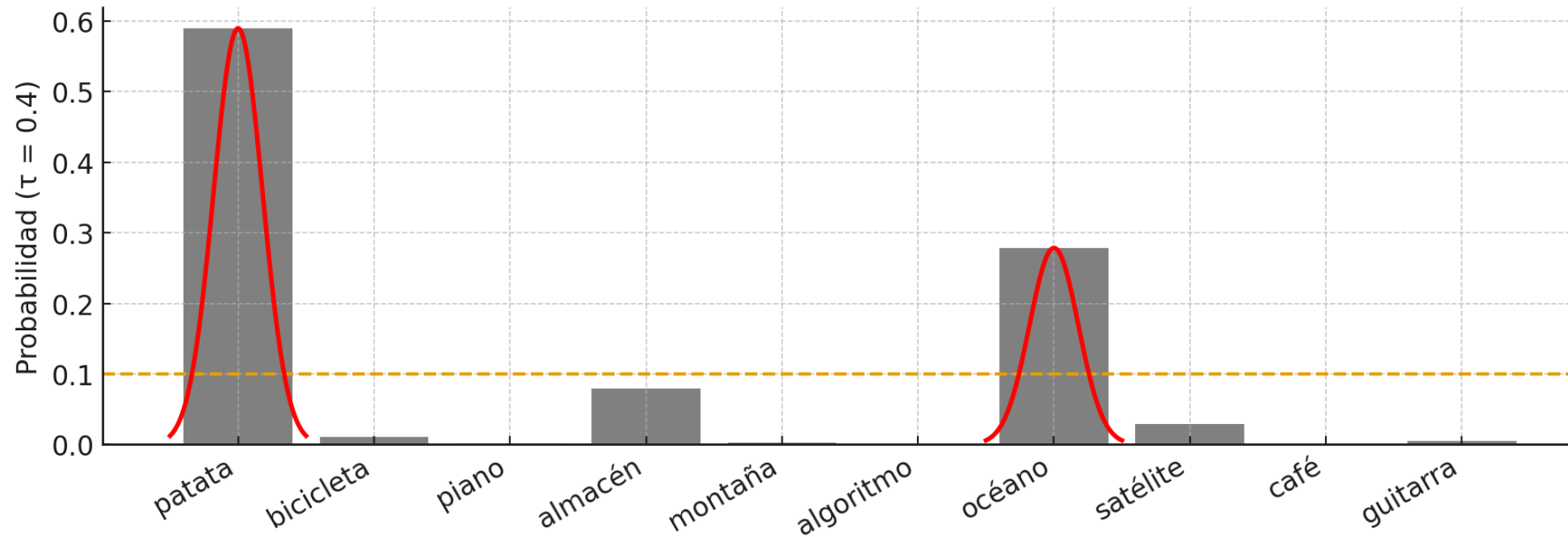
A diagram illustrating attention weights for the word "Los". Blue curved arrows originate from "Los" and point to each word in the sentence: "Me", "encantan", "los", "perros.", "Los", "prefiero", "a", "los", and "gatos.". The arrows vary in thickness. The thickest arrow points to "Los" itself, and other thick arrows point to "perros." and "gatos.", indicating high attention to these words.

Output del transformer

Depende del modelo, como veremos, pero el **output del decoder** suele ser un vector de probabilidades asociadas a *tokens* (por ejemplo, asociados a un vocabulario).



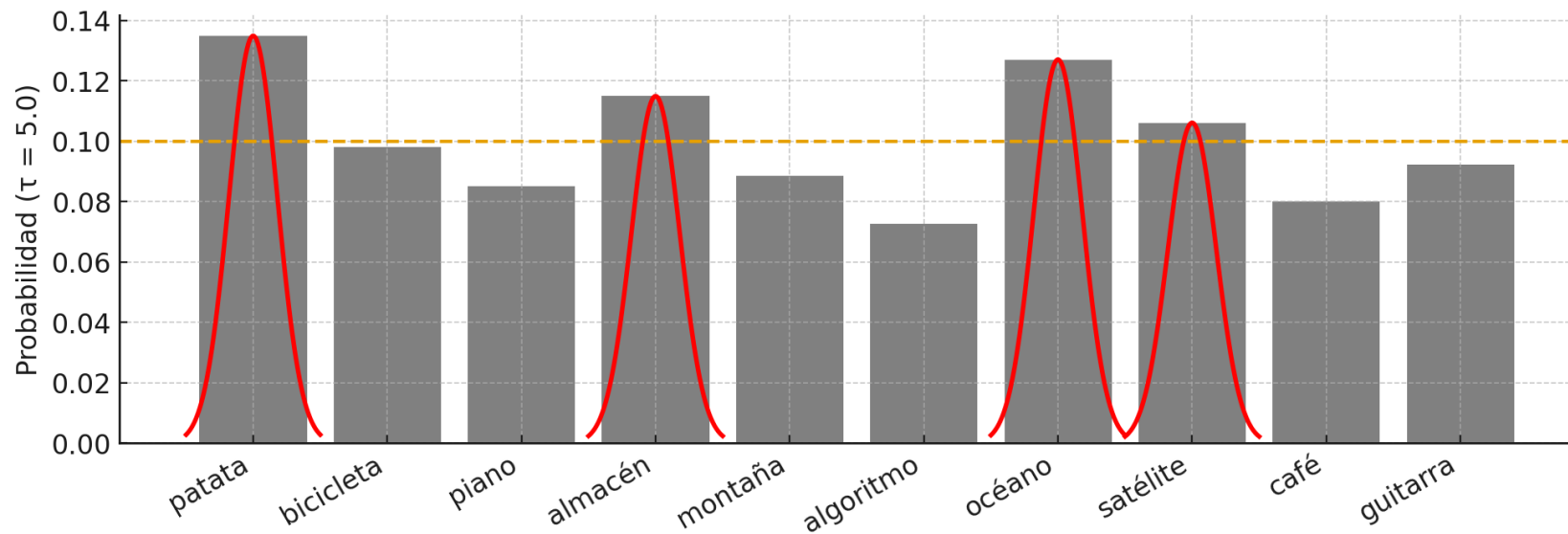
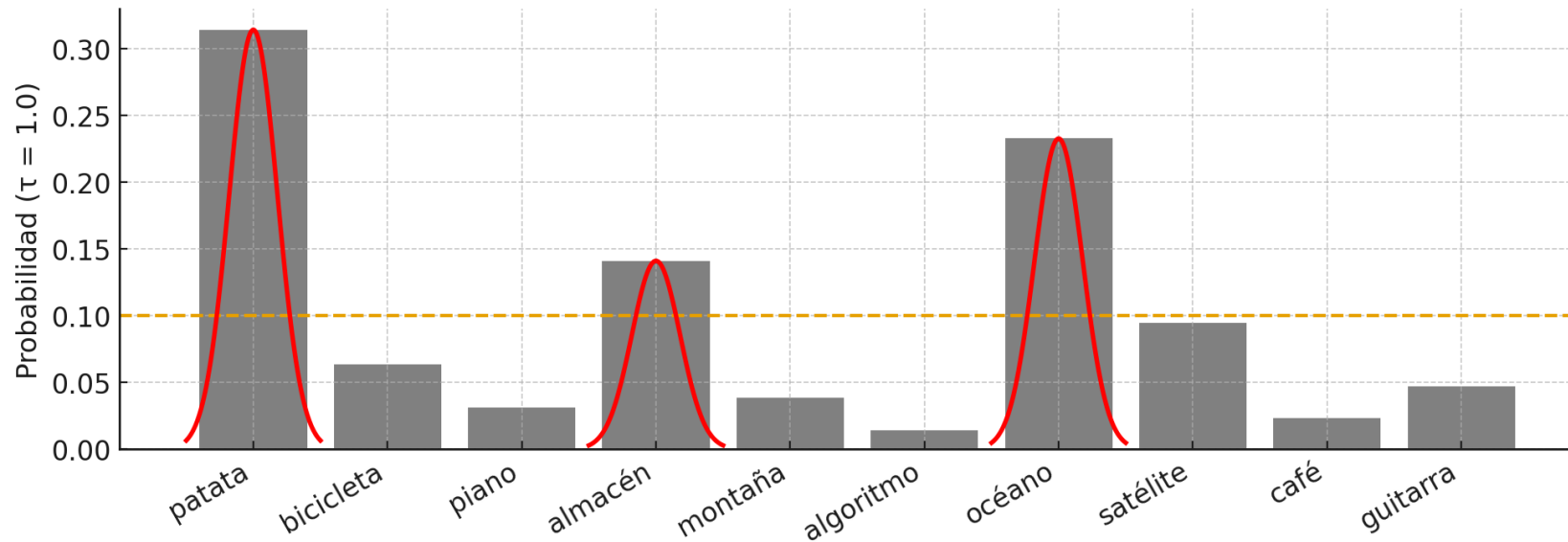
Softmax(logits) con baja temperatura.



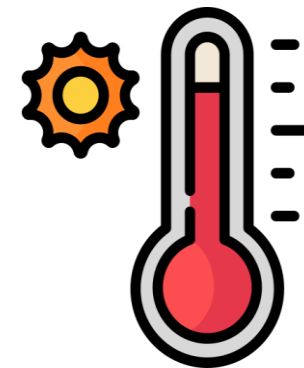
¿Cómo escogemos el siguiente *token*?

- Top-p: se establece un umbral y se muestrea según la probabilidad de cada *output*.
- Aún así, este muestreo hace que no sea determinista.



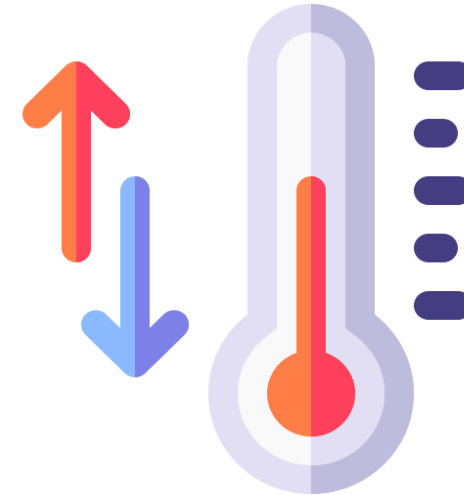


Ejemplos de temperatura
media y alta.



Softmax con *temperature* (τ):

$$p_i(\tau) = \frac{\exp\left(\frac{z_i}{\tau}\right)}{\sum_{j=1}^N \exp\left(\frac{z_j}{\tau}\right)}$$

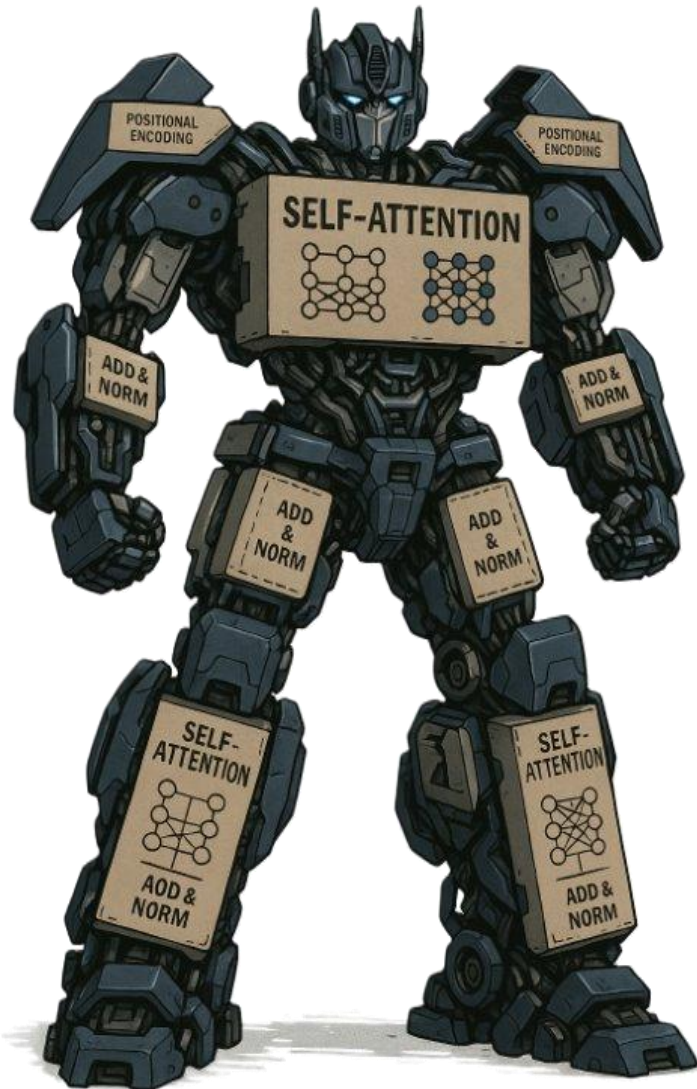


- Temperatura baja: más reproducibilidad, fiabilidad, coherencia.
- Temperatura alta: más creatividad, diversidad.



Arquitecturas de *transformers*





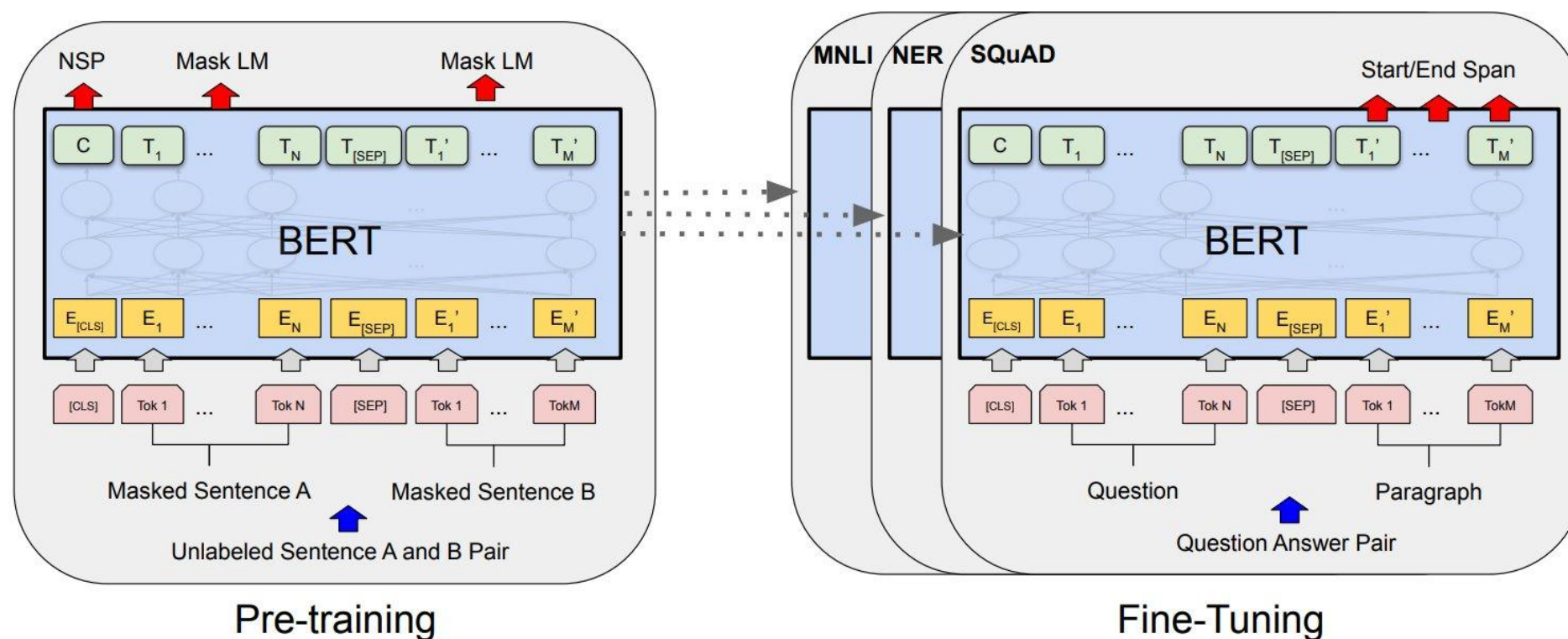
Vamos a ver:

- LLMs: grandes modelos del lenguaje. Arquitectura BERT y arquitectura GPT.
- Otros modelos con *transofmers*: visión, generación de audio, series temporales, etc.



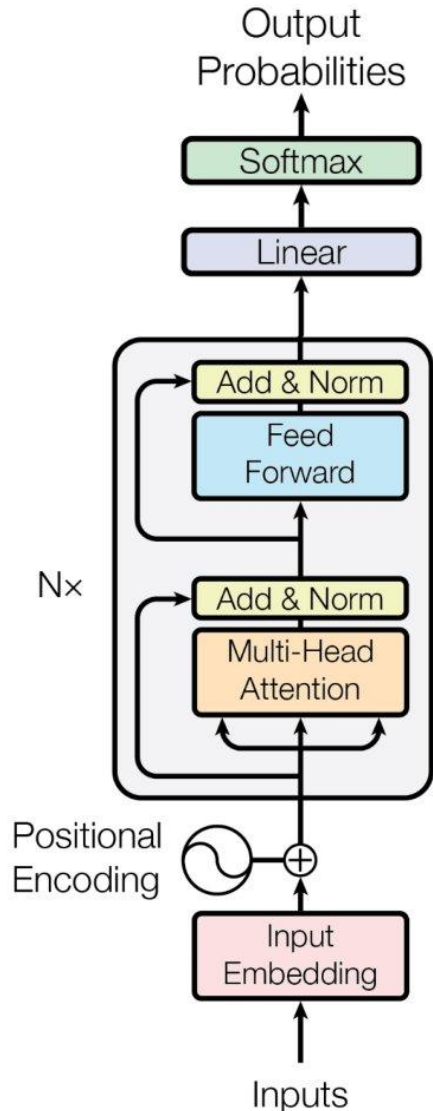
¿Qué es BERT?

Bidirectional Encoder Representations from Transformers. Es un encoder de transformer entrenado de manera bidireccional sobre grandes corpus.



BERT no genera texto, lo entiende.





Solo se usa el *encoder*:

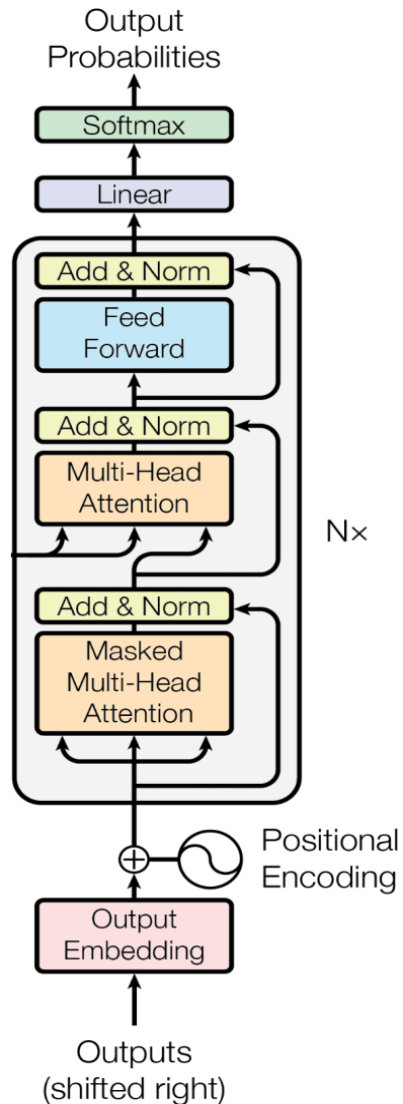
- **BERT-Base:** 12 capas, 12 cabezas, hidden 768, ~110M params.
- **BERT-Large:** 24 capas, 16 cabezas, hidden 1024, ~340M params.

Cientos de *fine tunings*: idioma, dominio específico y un amplio espectro de tareas:

- Clasificación.
- Reconocimiento de entidades (NER).
- Responder preguntas a partir de un texto (QA).
- *Matching* de pares de oraciones.
- Análisis de sentimiento.
- Etc...



¿Qué es GPT?



General Pretrained Transformer. Es un *decoder* de *transformer* entrenado de manera autorregresiva sobre un gran corpus de datos.

Tarea: predecir el siguiente *token* más probable. Capacidades emergentes. Se alimenta de los datos pasados y genera nuevos de manera recurrente.

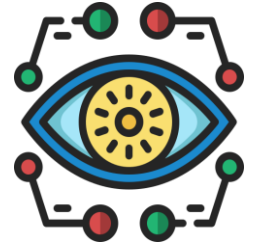
Aplicaciones:

- Generación de texto.
- Se entrena con aprendizaje por refuerzo para obtener un **chatbot**.
- También se puede entrenar para que realice tareas específicas.

GPT aprende a predecir la siguiente palabra.



- **Visión:** fundamentalmente modelos [ViT](#). Se tokenizan *patches*. *Patches* multiescala ([Swin](#), [PVT](#)). También se usan con *embeddings* multimodales ([CLIP](#)). Modelos de segmentación "promptable" como [SAM/SAM-2](#). Los *diffusion transformers* ([DiT](#)) usan la idea de difusión pero se sustituye la U-Net por un backbone ViT.
- **Audio:** ASR ([whisper](#)), generación de música (como la base de [Suno](#), [Udio](#)...). Se usa [EnCodec](#) para tokenizar audios. Se generan *tokens* de audio de manera autorregresiva ([MusicGen](#)) o no autorregresiva ([SoundStorm](#)).
- **Series temporales:** modelos base para forecasting de energía, finanzas, meteorología... ejemplo: [Informer](#), [Autoformer](#)... algunos modelos fundacionales: [TimesFM](#) (Google), [Chronos](#) (Amazon).
- **Otras aplicaciones:** uso de grafos para estructura de materiales ([MatterGen](#)). Uso de LLMs donde los *tokens* son aminoácidos / bases ([proteínas](#), [ADN/ARN](#)).



Aplicaciones de LLMs



¿Cómo aplicamos estos modelos?

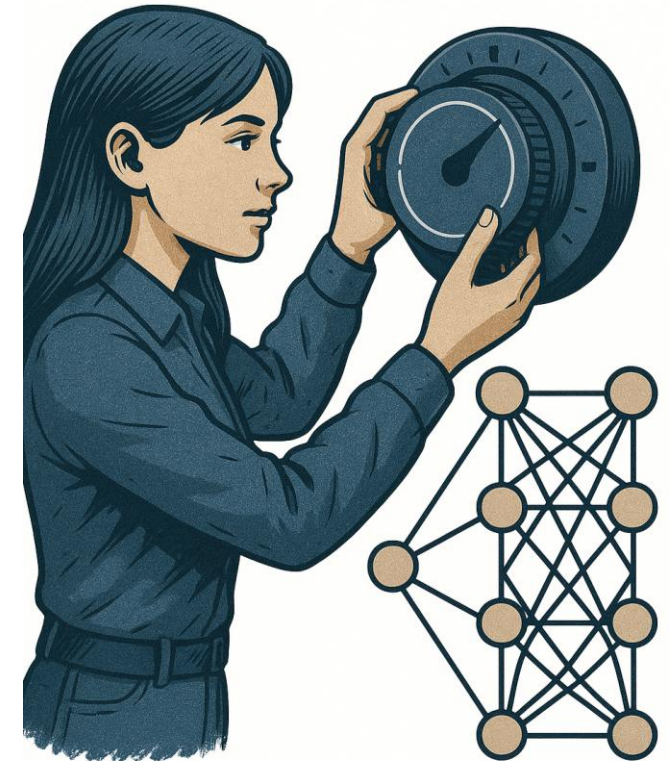
Pasamos de "predice el siguiente *token*" a una aplicación real:

- *Fine tuning* (dominio específico).
- RL (tarea específica).

Métodos para diferentes aplicaciones:

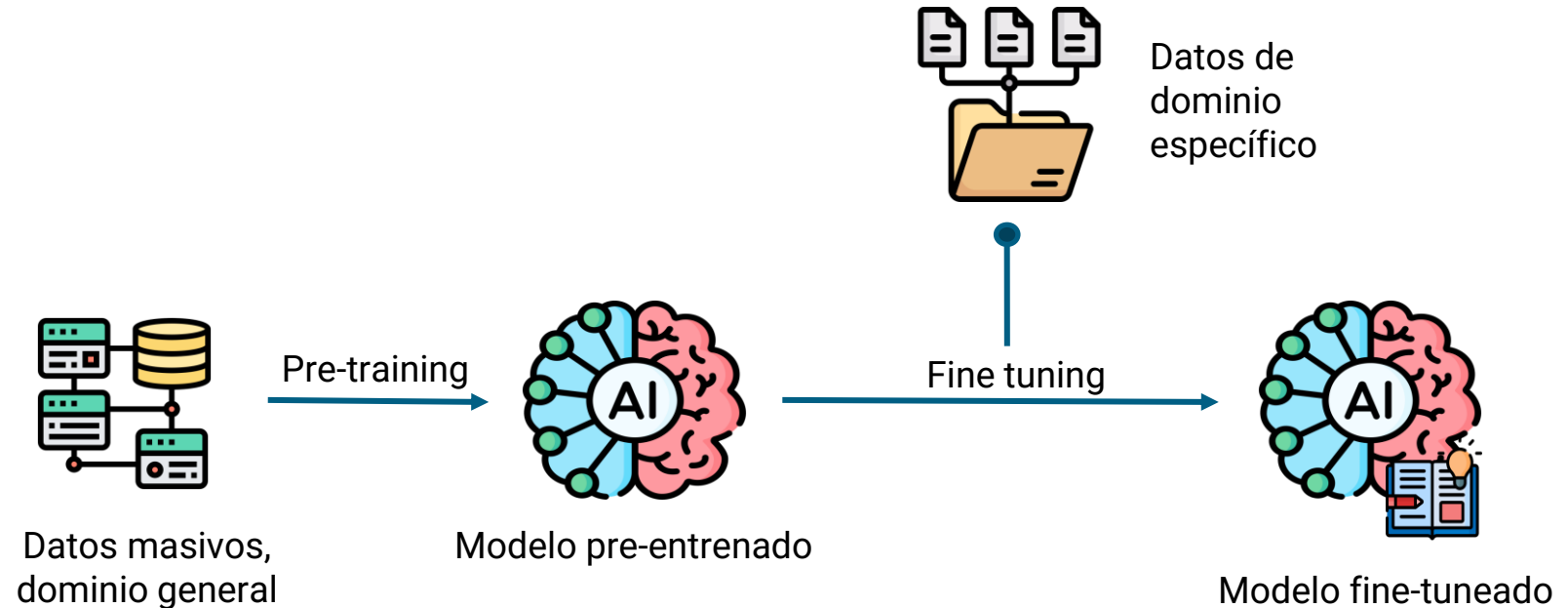
- *Prompting*.
- RAG.
- *Test-time compute*.
- Multimodalidad.
- Herramientas.

FINE-TUNING



Muy usado en modelos tipo BERT: otros idiomas, dominios específicos, etc.

En modelos tipo GPT el *fine tuning clásico* es delicado, se usa en casos concretos pero no suele ser la mejor alternativa.



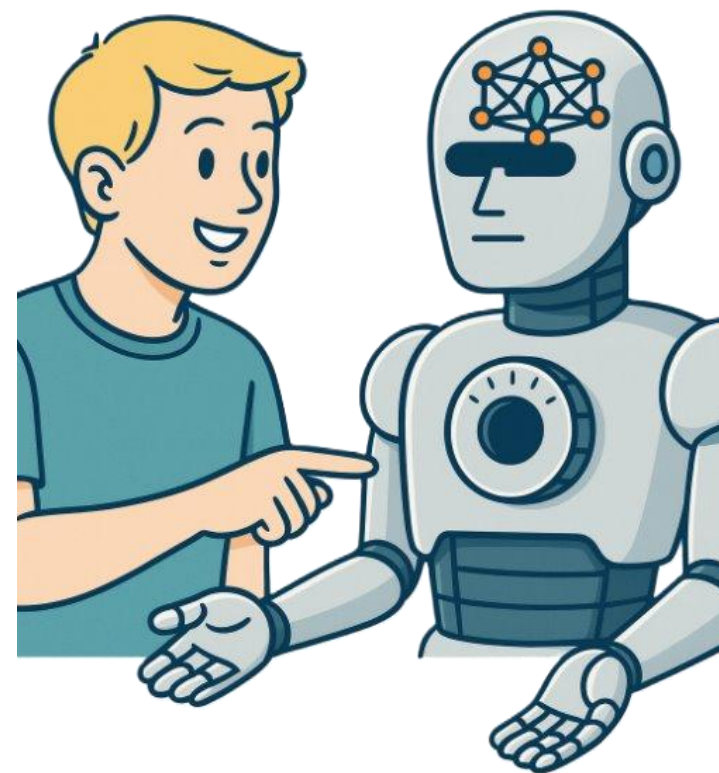
Es más habitual usar otras estrategias como LoRA (*Low-Rank Adaptation*): se añaden y entrenan solo unos pocos parámetros para que aprenda cierta información.

En RL tradicional, el agente aprende a partir de recompensas / castigos cuál es la política óptima, explorando diferentes estados y acciones.



- **RLHF:** se aprende a partir de preferencias de personas.
- **SFT** (*supervised fine tuning*): usando pares prompt-respuesta redactados por personas.
- **Reward modeling:** pares de respuestas para un mismo prompt. Se favorece una.
- **RLAIF:** RL con AI *feedback*.

RLHF



Un LLM de generación modela $p(\text{texto}|\text{contexto})$. El *prompt* fija el **condicionamiento** (lo que el modelo “ve”) y por tanto **moldea la distribución** de sus siguientes *tokens*, dada por p .

Para un LLM "base", el *prompt* típicamente refleja una serie de ejemplos de la tarea a realizar.

Para un *chatbot*, el *prompt* refleja instrucciones. Puntos clave:

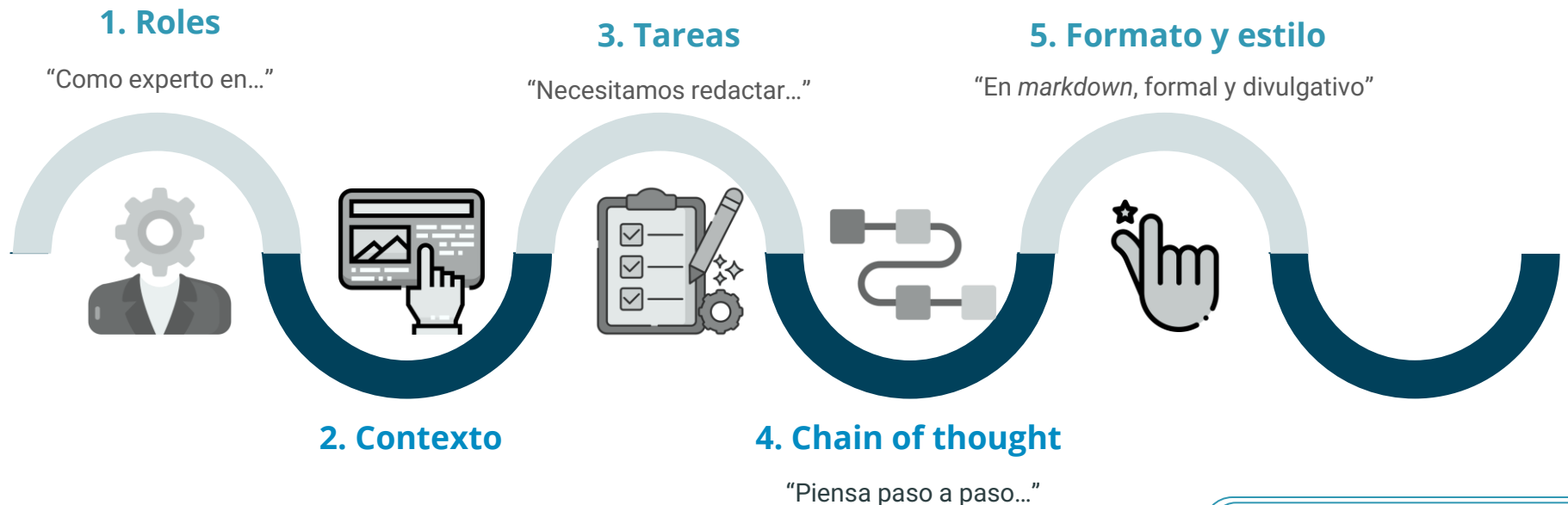
- Roles.
- Contexto.
- Tarea.
- *Chain-of-thought*.
- Formato y Estilo.



Un LLM de generación modela $p(\text{texto}|\text{contexto})$. El *prompt* fija el **condicionamiento** (lo que el modelo “ve”) y por tanto **moldea la distribución** de sus siguientes *tokens*, dada por p .

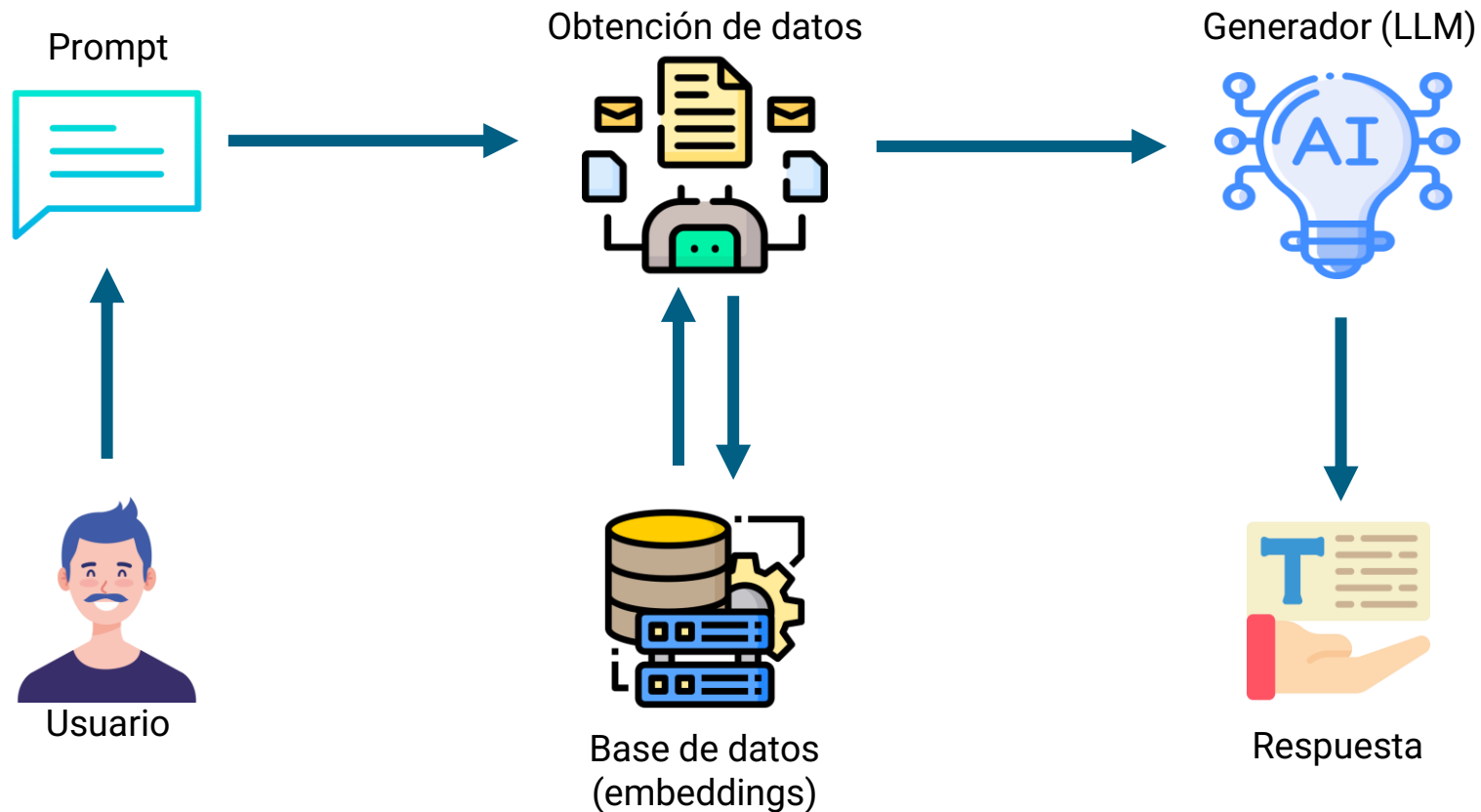
Para un LLM "base", el *prompt* típicamente refleja una serie de ejemplos de la tarea a realizar.

Para un *chatbot*:



Retrieval Augmented Generation. Idea clave: **Recuperar Antes de Generar**.

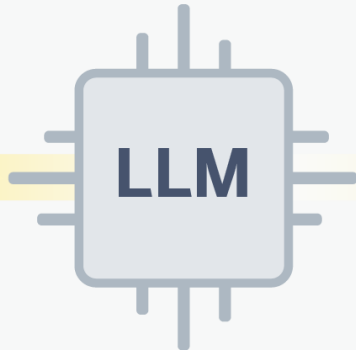
Es la manera de conectar al LLM con documentación externa.



Prompting

Pompt

Crea una historia sobre un gato



SALIDA



Esta es la historia de un gato que solía jugar al baloncesto

Fine-tuning

Ejemplos de historias de gatos



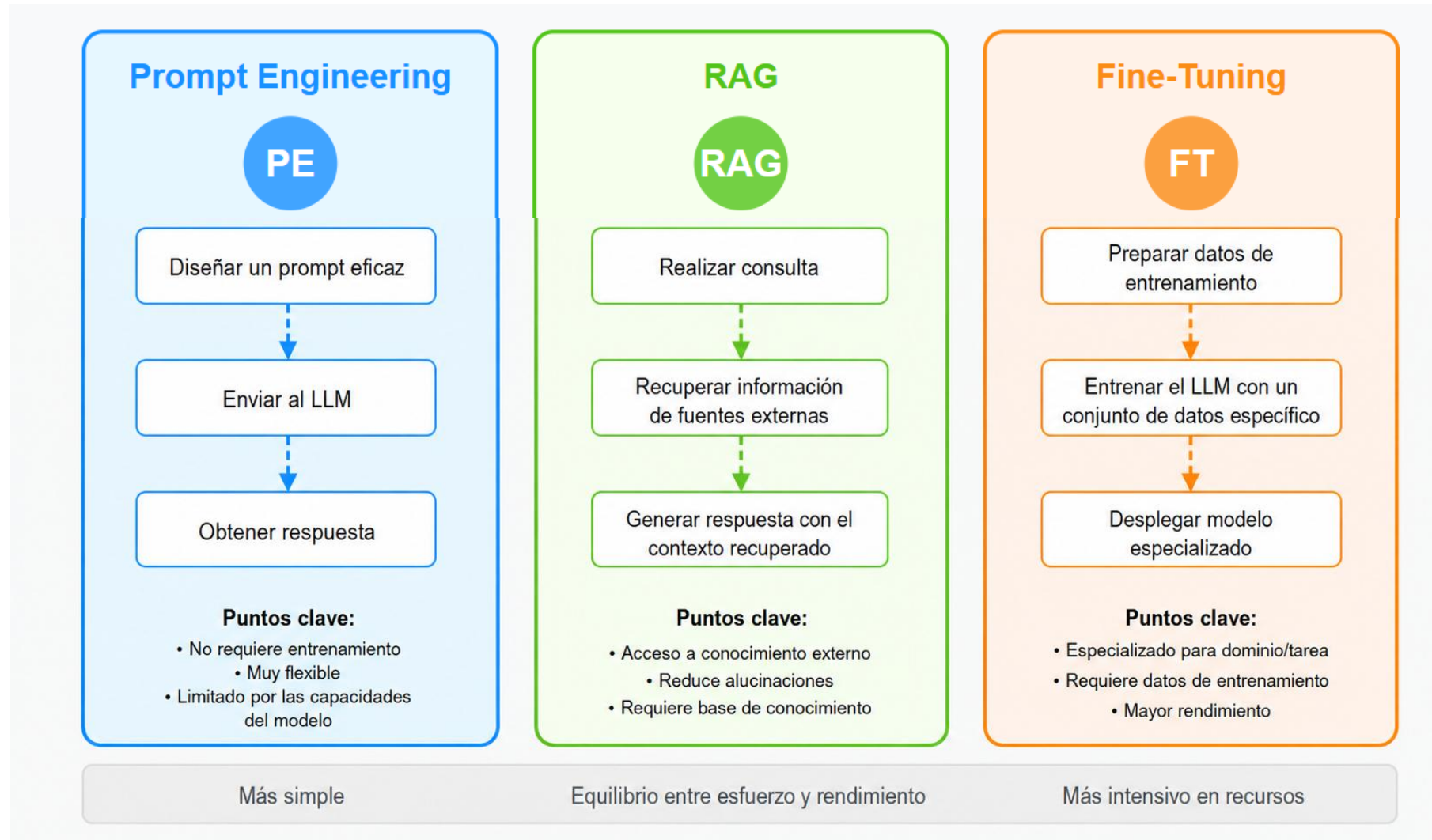
Fine-tuning data



SALIDA

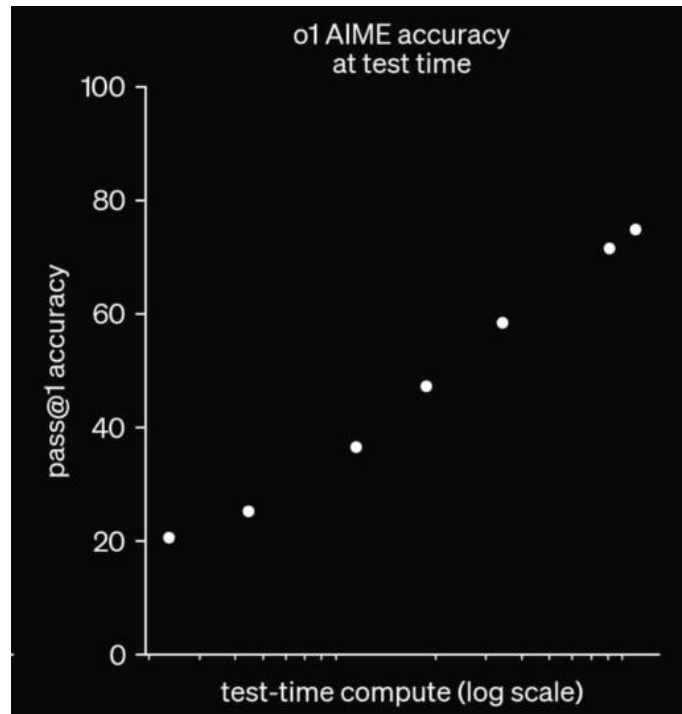


Al entrenar el modelo con muchos ejemplos similares, éste se especializa en historias de gatos



Un LLM, tras recibir un *prompt* de entrada, no genera directamente *tokens* de salida:

- Paso intermedio en el que genera *tokens* "de pensamiento".
- A partir de estos *tokens*, genera una respuesta.



Test-time compute:

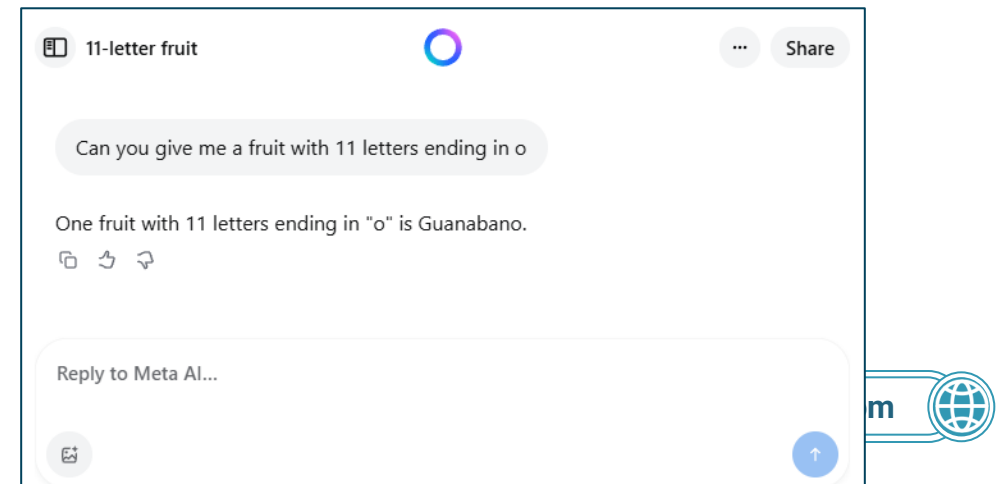
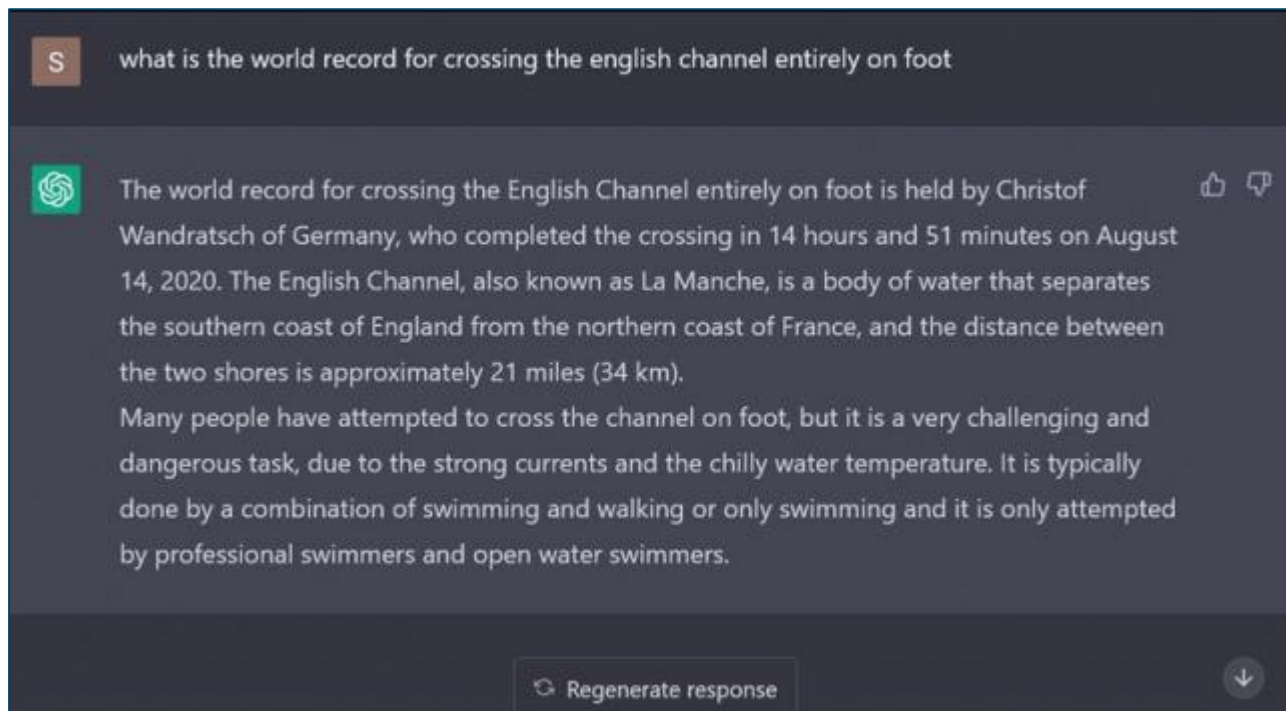
- El modelo dedica más tiempo / *tokens* a "pensar" y/o realizar pruebas.
- Solo da una respuesta tras esta fase de entrenamiento.
- Para que funcione adecuadamente, hace falta incluir este comportamiento en el RL – modelos razonadores.
- Balance entre tiempo/coste y precisión.





Los modelos pueden inventarse cosas y sonar muy convincentes.

- El modelo no sabe si algo es verdad o mentira, solo predice texto probable.
- GPT no busca la verdad, busca la siguiente palabra más probable.
- Con métodos como RAG se soluciona ese problema porque consulta datos reales.



Prompting y ejemplos





EBIS Business
Techschool

www.ebiseducation.com

